

Software Construction I

VidToCode



Where every word in a tutorial
becomes a line of working code.

Ahmed Bahaj, Abdullah Misar,
Yazeed Bafaqih, Albaraa Alnahari

Supervised by: Dr. Ahmed Tayeb
& Dr. Mohammed Alahmadi

17TH February, 2026



Optimizing OCR Performance for Programming Videos



جامعة جدة
University of Jeddah



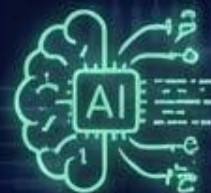
Problem:

While OCR works well for normal text, programming code contains special symbols, indentation, and syntax that make it difficult to recognize accurately.



Result:

Their results show that OCR accuracy for code is often around 60%, and performance drops significantly for dark themes and small fonts.



→ CodeT5-OCRfix

Impact:

This study highlights that OCR alone is not sufficient for reliable code extraction, motivating the need for AI-based correction models like CodeT5-OCRfix.

Extracting Code from Programming Tutorial Videos



جامعة جدة
University of Jeddah



Problem:

Extracting accurate source code from programming tutorial videos. OCR tools like Tesseract often produce many errors because code has strict syntax and special characters.



Methodology:

The authors propose CodeT5-OCRfix, a system that uses a fine-tuned code-aware language model, CodeT5, to correct OCR errors.



Key Contribution:

Their results show significant improvements over traditional OCR and statistical correction methods, both in accuracy and speed.

Case2Code: Scalable Synthetic Data for Code Generation

Goal:

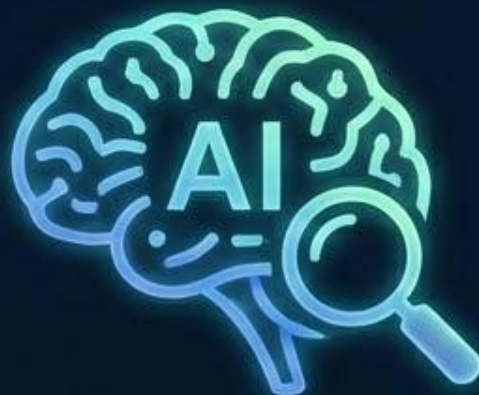
A new inductive reasoning task for large language models. Instead of generating code from instructions, Case2Code requires models to infer the original program from input-output examples.



Input



Output



Large Language Model
(Inductive Reasoning)



Original Program

Case2Code: Scalable Synthetic Data for Code Generation

Approach:



Collect Real Python Functions
(from large datasets)



Generate Example Inputs
(using small LLM)



Execute Function
(Python Interpreter)



Obtain Correct
Outputs

Impact:



This work demonstrates that large-scale synthetic inductive data can improve reasoning abilities in code LLMs.

CodeGen2: Lessons for Training LLMs on Programming and Natural Languages

Objective:

Make AI code generation cheaper by **unifying** 4 pieces, model design, **training method**, code-completion ability and data mix.



Methodology:

Test small 1B models with different mixes of the 4 pieces. Best mix is then scaled up to **16B models**.



Key Finding:

Simpler models work as good as complex models, Re-training on same data multiple times boosted performance.



Evaluating Large Language Models Trained on Code



Objective:

Foundational paper for Codex, train it to write working python functions from English descriptions (docstrings)



docstring



python function



Fine-tune



Methodology:

Fine-tune GPT on public github python code, tested by generating multiple code attempts per problem.



Public GitHub
Python Code



GPT Model



Key Finding:

1 attempts -> 28.8% success rate,
100 attempts -> 72.3% success rate



A Deep Learning Approach to Identifying Source Code in Images and Video



Objective:

Build AI model to distinguish frames with code and frames with slides, and group similar code examples together.



Methodology:

Collected 22 hours of tutorials, trained VGG16 to detect visible code and handwritten ones.



Key Finding:

375x faster than humans at labeling frames, reducing OCR waste by only processing frames that actually contain code.

SeeHow: Workflow Extraction from Programming Screenscasts

Problem



Programming screenscasts contain valuable coding workflows, but existing approaches only capture low-level interactions or coarse video segments, making it difficult to understand how code evolves step by step.

Methodology



Computer vision for detecting code regions



Action recognition to identify editing operations (typing, deleting, selecting)



Text extraction to reconstruct code snippets



The system converts video frames into structured coding workflow steps.

Impact



This work enables automatic extraction of line-level coding actions, improving accessibility and helping developers understand programming workflows from tutorial videos.

Improving Code Extraction Using a Code-Aware Encoder-Decoder Model

Methodology: CodeT5-OCRfix



Problem

Traditional OCR tools struggle with programming code because of strict syntax, indentation, and special characters, leading to inaccurate code extraction from screencasts.



Scan the screen of eneast syntax, sitrict inderrication, and inaccurate code extraction from screencasts.



CodeT5 Model

Applies a fine-tuned CodeT5 language model to corrects OCR errors a deminction from structure



Clean a nean code block wildres the corrects OCR errors charanty using code semantics and structure.



Result

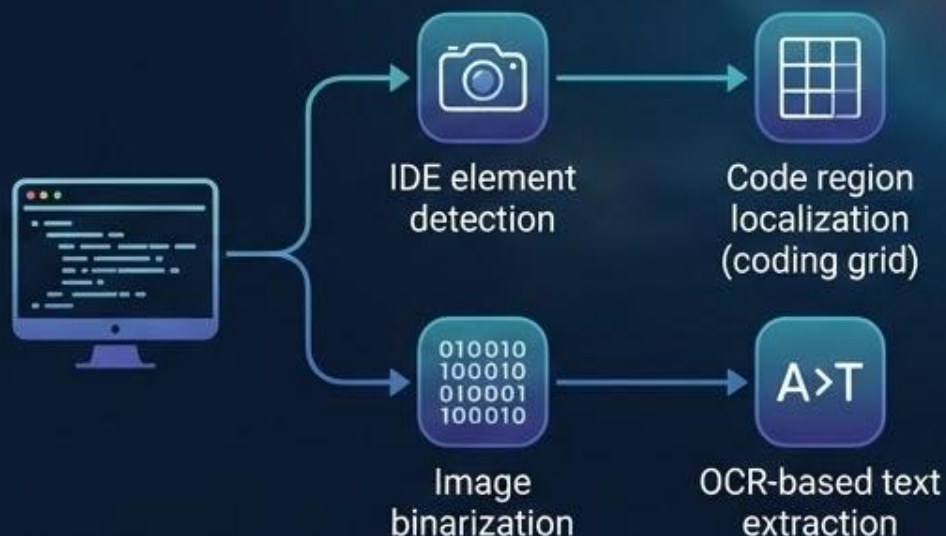
The proposed approach significantly improves code extraction accuracy and speed compared to traditional OCR and statistical correction methods.

CodeSCAN: ScreenCast ANalysis for Video Programming Tutorials



Problem

Programming tutorials are mainly video-based, making it difficult to search for specific code snippets. Extracting accurate source code from IDE screenshots is challenging due to complex layouts, themes, fonts, and visual artifacts.



Methodology

The authors introduce **CodeSCAN**, a large-scale dataset of 12,000 IDE screenshots with detailed annotations. The system pipeline includes: [bullets].



Impact

CodeSCAN provides the first comprehensive benchmark for screencast code extraction, enabling better search, indexing, and automated analysis of programming tutorial videos.

Code Completion with Statistical Language Models



Problem

Developers often struggle with complex API usage and method sequences. Traditional autocomplete tools lack contextual understanding and cannot generate complete method sequences.

Result

The system achieves high accuracy, with the correct completion appearing in the top three suggestions in over 90% of cases.



Impact

This work established that code is predictable and repetitive, laying the foundation for modern AI-based code assistants.



Methodology

The authors treat source code as natural language.



- Extract method call sequences from large codebases
- Train statistical language models (N-gram and RNN)
- Predict the most probable method sequence to complete missing code

Conclusion



Statistical Models
(Foundation)



Deep Learning
(Complex Patterns)



LLMs
(Inductive Reasoning,
Code Understanding)



VidToCode leverages these advancements to turn passive video watching into an active, code-rich experience.

Thank You